Agentic LLM Ranking versus Trained Sequential Models: An Audited Study of ARAG, GRU Fusion and Text-Aware Sequence Modelling on Amazon Reviews 2023 (Video Games)

[Author Name]
[Course / Programme — Deep Learning Project]
[Institution]

October 2026

Abstract

Large language models (LLMs) can rank recommendation candidates by reasoning over a user's history in natural language. ARAG [12] organises this reasoning as a pipeline of four cooperating agents. We implement an ARAG-style recommender on the Amazon Reviews 2023 Video Games category [6] and ask whether a *trained* deep-learning component — a GRU sequential model [4] — improves it when late-fused into the LLM ranking. To isolate ranking from retrieval, every system ranks the same frozen 20-item candidate list per user (the held-out next item plus 19 popularity-matched negatives), and all contrasts are tested with a paired bootstrap. With `gpt-4o-mini`, ARAG reaches NDCG@5 = 0.424 on 100 test users and significantly beats the paper's LLM baselines, the ID-only GRU, popularity and random ranking; the paper's NLI+CSA ablation also reproduces significantly. Fusing the ID-only GRU does *not* help (H1: $\Delta = -0.013$, 95% CI $[-0.032, +0.005]$). We trace the GRU's weakness to sparse histories (median four training items per user) and introduce a *text-aware GRU* whose item vectors combine a learned ID embedding with a projected, frozen MiniLM embedding of the item's metadata. With the same architecture, configuration and training budget, it raises the trained model from NDCG@5 0.279 to 0.461 on all 94,762 test users, significantly outperforms content-similarity and ItemKNN baselines, and is numerically above ARAG on the 100-user sample (0.462 vs. 0.424, not significant). The extension required zero new LLM calls. We also document a protocol flaw, found in review, that had produced a false positive result, and we release an audit pipeline that re-derives every reported number from raw predictions.

Keywords: recommender systems; large language models; agentic retrieval-augmented generation; sequential recommendation; GRU4Rec; late fusion; evaluation methodology.

Contents

1 Introduction **3**

2 Related Work **3**

3 Data **4**

4 Evaluation Protocol **5**
4.1 Frozen candidate lists . . . 5
4.2 Users . . . 5
4.3 Metrics . . . 5
4.4 Statistical testing . . . 5

5 Methods **6**
5.1 System overview . . . 6
5.2 ARAG agent pipeline . . . 6
5.3 GRU sequential model . . . 7
5.4 Text-aware GRU . . . 7
5.5 Non-learned baselines . . . 8
5.6 Late fusion . . . 8

6 Experimental Setup **8**

7 Results **9**
7.1 Main comparison . . . 9
7.2 RQ1: does fusing the GRU help ARAG? . . . 12
7.3 RQ2: do the paper's claims hold? . . . 12
7.4 RQ3: the text-aware GRU . . . 12

8 Discussion **14**
8.1 Why the ID-only GRU adds nothing to ARAG . . . 14
8.2 The bottleneck was representation, not model class . . . 15
8.3 How much of ARAG's advantage is agentic reasoning? . . . 15
8.4 Cost–quality trade-off . . . 15
8.5 A methodological lesson: candidate construction can manufacture conclusions . . . 15

9 Threats to Validity **16**

10 Conclusion and Future Work **16**

A Worked Example **18**

B Errata: the Superseded Run **18**

C Reproducibility **18**

1 Introduction

Sequential recommendation predicts the next item a user will interact with from the order of their past interactions. For a decade the standard tools have been neural sequence models trained on interaction logs, from recurrent networks such as GRU4Rec [4] to self-attentive models such as SASRec [10]. Large language models offer a different route: they can read item titles and descriptions, infer a user's intent in natural language and rank candidates zero-shot [7]. Agentic pipelines push this further by splitting the task into specialised LLM calls. ARAG [12] uses a *User Understanding Agent*, a *Natural Language Inference* agent, a *Context Summary Agent* and an *Item Ranker Agent*, and reports large gains over single-prompt LLM baselines.

The two families have complementary strengths. Trained models capture collaborative co-occurrence patterns across hundreds of thousands of users, but see only opaque item IDs; LLMs understand item content but see only one user at a time and cost several API calls per recommendation. This project asks whether combining them helps, and what each contributes.

Research questions.

RQ1 On identical frozen candidate lists, does late-fusing a trained GRU into ARAG's ranking improve over ARAG alone (**H1**) and over the GRU alone (**H2**)?

RQ2 Do the base paper's claims — ARAG beats simpler LLM baselines, and each agent component contributes — hold on a new category and model?

RQ3 Why does the trained model under- or over-perform, and can a representation change close the gap without additional LLM cost?

Contributions.

1. An ARAG-style four-agent recommender with strict JSON-schema outputs, a repairing ranking parser, and a disk cache that makes every LLM run exactly replayable at zero cost.
2. A ranking-isolating evaluation protocol: frozen 20-item lists with popularity-matched negatives, a single shared user sample for all systems, validation-only tuning and paired bootstrap confidence intervals.
3. A negative result for RQ1 with an ID-only GRU (H1 fails), together with a statistically significant reproduction of the base paper's baseline ordering and NLI+CSA ablation (RQ2).
4. A *text-aware GRU* that adds frozen sentence embeddings of item metadata to the item representation, lifting the trained model by +0.174 NDCG@5 on all 94,762 test users and matching or exceeding the agentic pipeline at a fraction of its cost (RQ3).
5. A documented methodological lesson: an earlier candidate-list construction made popularity-like systems look dominant and produced a spurious "H1 supported" result. We report the diagnosis, the fix and both sets of numbers.

The rest of the report is organised as follows. Section 2 reviews related work, Section 3 describes the data and Section 4 the evaluation protocol. Section 5 presents the systems, Section 6 the experimental setup and Section 7 the results. Section 8 discusses them, Section 9 lists threats to validity and Section 10 concludes.

2 Related Work

Sequential recommendation. GRU4Rec [4] introduced recurrent networks [1] for session-based recommendation, trained with a next-item objective. SASRec [10] replaced recurrence

with causal self-attention and is today a standard strong baseline. Both models represent items by learned ID embeddings, so items with few interactions receive poorly trained vectors — a weakness that becomes acute when user histories are short.

Text-based and transferable sequence models. Several works replace or augment ID embeddings with representations of item text. UniSRec [5] learns universal item representations from text with a pre-trained language model so that sequence models transfer across domains. Our text-aware GRU follows the same principle in its simplest form: a frozen sentence encoder [13, 15] supplies item features that a learned linear projection maps into the GRU's embedding space, added to the ID embedding.

LLMs as recommenders. LLMs can act as zero-shot rankers when given a user's history and a candidate list in the prompt [7]. Retrieval-augmented variants select which parts of the history to show. ARAG [12] decomposes ranking into user understanding, candidate–user natural language inference, context summarisation and final ranking, and reports gains of 25–42% NDCG@5 over recency-based and vanilla RAG prompting on three Amazon categories.

Evaluation methodology. Sampled-candidate evaluation is cheap but can distort system rankings, depending on how the negatives are drawn [11]. Strong, well-tuned simple baselines often match complex neural recommenders [2], which motivates our inclusion of content-similarity and item-based collaborative filtering [14] baselines. We use the paired bootstrap [3] for significance, with NDCG [8] as the primary metric.

3 Data

We use the Amazon Reviews 2023 corpus [6], Video Games category, in its official rating-only 5-core leave-last-out benchmark split. For each user, the last interaction is the test target, the second-to-last is the validation target and the remainder is training history. Item text comes only from product metadata (title, main category, categories, features and description, truncated to 120 words); review text is never used. Table 1 summarises the data.

Table 1: Dataset statistics (Amazon Reviews 2023, Video Games, 5-core leave-last-out).

Users	94,762
Interactions (train / valid / test)	625,062 / 94,762 / 94,762
Catalogue items with metadata	137,269
Items with at least one interaction	25,612 (18.7%)
Items in the training vocabulary	25,527
Training items per user: mean / median / 75th percentile	6.6 / 4 / 7
Raw files verified by SHA-256	4

Figure 1 shows that user histories are very short: 63.5% of users have only three to five training interactions. This property turns out to be central to our results (Section 8.2).

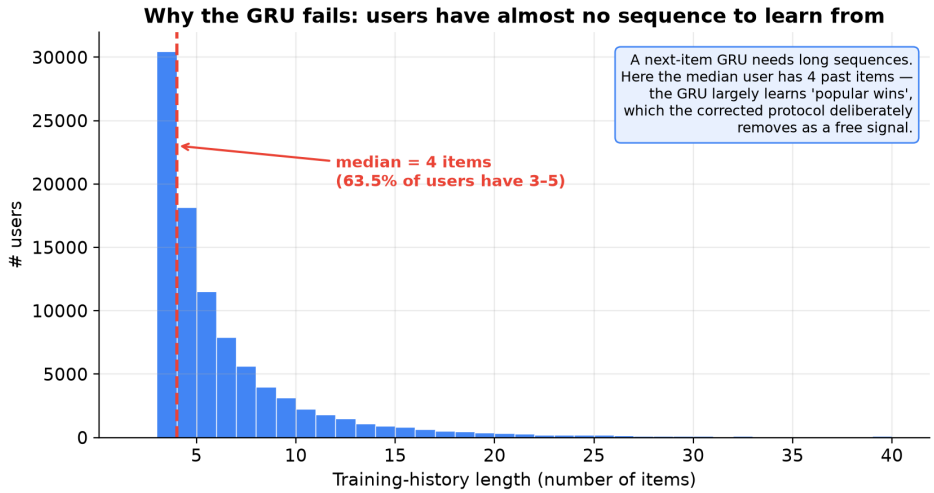


Figure 1: Distribution of training-history length per user. The median user has four training interactions, which gives an ID-only sequence model very little signal.

4 Evaluation Protocol

4.1 Frozen candidate lists

For each validation and test user we build one 20-item candidate list: the true held-out target plus 19 negatives sampled without replacement (seed 42) from the 25,527 training-vocabulary items, with probability proportional to training interaction count, excluding everything the user interacted with in any split. The list is shuffled, hashed and frozen before any system is evaluated. Every system ranks exactly this list.

This design isolates *ranking* from *retrieval*: if each system retrieved its own candidates, a retrieval miss would score zero and the comparison would conflate the two stages. Popularity-proportional sampling makes negatives roughly as popular as targets (on average 1.01× on validation and 1.40× on test), so global popularity carries no free signal. A uniformly random permutation has expected Hit@5 of exactly 0.25, a useful sanity anchor.

4.2 Users

LLM systems are expensive, so all systems are compared on the *same* 100 randomly sampled test users (and 100 validation users for tuning). Non-LLM systems are additionally evaluated on all 94,762 test users, which gives near-deterministic estimates for them.

4.3 Metrics

With a single relevant item per user at rank $r \in \{1, \ldots, 20\}$,

$$\text{Hit@}K = \mathbb{1}[r \le K], \qquad \text{NDCG@}K = \frac{\mathbb{1}[r \le K]}{\log_2(r+1)}, \qquad \text{MRR} = \frac{1}{r}, \tag{1}$$

averaged over users. Hit@5 and NDCG@5 are primary; we also report Hit@10, NDCG@10 and MRR.

4.4 Statistical testing

For two systems A and B evaluated on the same users we resample users with replacement 10,000 times and report the mean per-user difference with its 95% percentile interval [3]. A contrast is called significant when the interval excludes zero. All hyperparameters are tuned on validation users only; the test set is never used for any selection.

5 Methods

5.1 System overview

Figure 2 shows the overall architecture. A data layer produces sequences and frozen lists; three families of rankers score each list; a fusion layer optionally blends an LLM ranking with a trained-model ranking; and an evaluation layer computes metrics and bootstrap intervals from the saved per-user rankings.

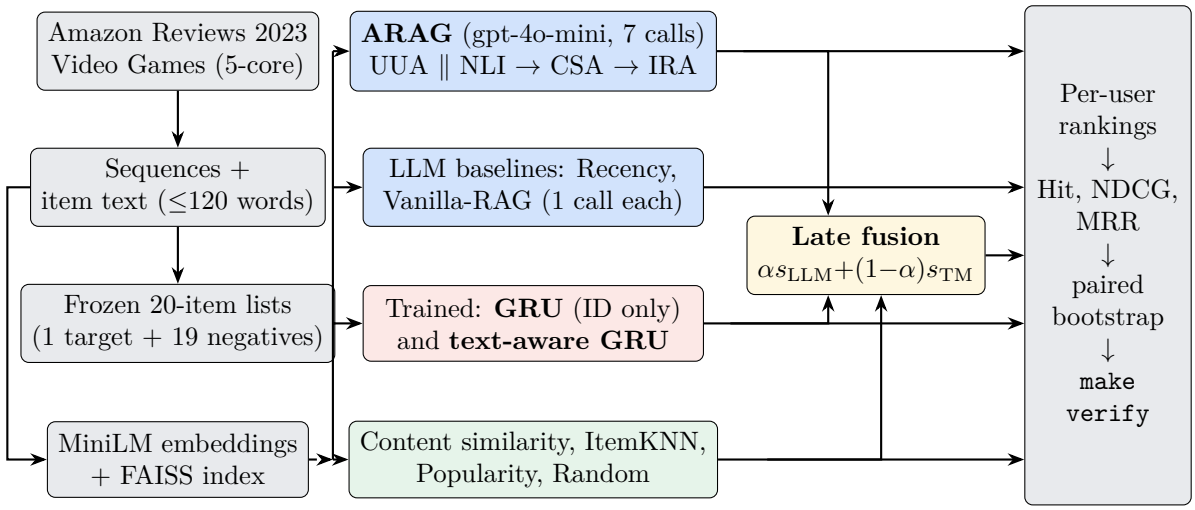


Figure 2: System architecture. All rankers score the same frozen list. The fusion layer blends the ARAG ranking with a trained-model or content ranking (s_{TM}); MiniLM embeddings feed the content baseline and, as item features, the text-aware GRU.

5.2 ARAG agent pipeline

Figure 3 details the agentic ranker. Each user's history (training items plus, for test, the validation item) is rendered as lines of the form *title* | *category* | *rating* | *date*. The long-term context C_{lt} holds up to 50 items and the session context C_{st} the last five.

- **UUA** (User Understanding Agent, 1 call): from C_{lt} and C_{st}, produces interests, current intent, dislikes and a summary of at most 120 words, S_{user}.
- **NLI** (4 calls of 5 candidates): for every candidate, a label (*support*/*neutral*/*contradict*), a compatibility score in $[0, 10]$ and a rationale, judged against the raw history.
- **CSA** (Context Summary Agent, 1 call): summarises the accepted candidates $\mathcal{I}^{+} = \{i : \mathrm{score}_i \geq \theta\}$ in at most 200 words, S_{ctx}. If no candidate passes θ, the top three by score are used and the fallback is counted.
- **IRA** (Item Ranker Agent, 1 call): given S_{user}, S_{ctx} and all 20 candidates, outputs a full permutation π_{LLM} with reasons.

UUA and NLI run in parallel. All outputs are requested as strict JSON schemas and validated. A ranking parser repairs malformed permutations (drops unknown IDs, removes duplicates, appends missing IDs by NLI score) and counts every repair. Two ablations follow the paper: *ARAG w/o NLI* (CSA summarises all 20 candidates) and *ARAG w/o NLI & CSA* (IRA receives only S_{user} and the raw candidates).

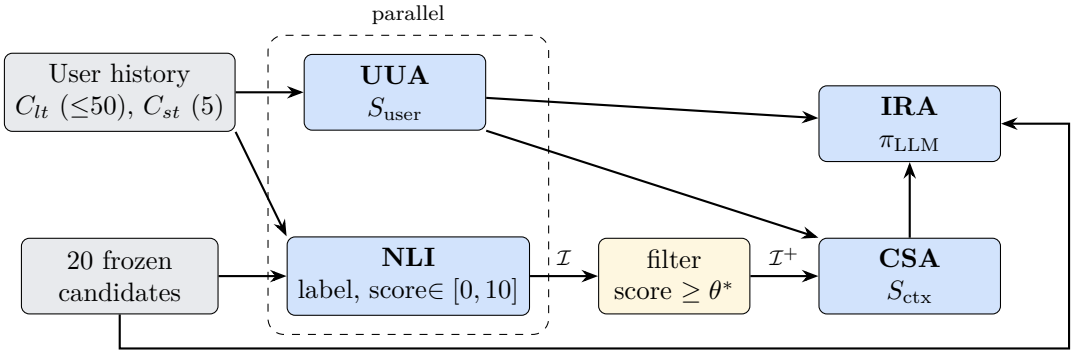


Figure 3: ARAG agent pipeline (seven LLM calls per user). $\theta^* = 5$ is tuned on validation.

5.3 GRU sequential model

Our trained model is a GRU4Rec-style network [4]. Let $\mathcal{V}$ be the training vocabulary (index 0 = padding, 1 = out-of-vocabulary). For a history $(i_1, \ldots, i_T)$, truncated to the last $T \leq 50$ items,

$$\mathbf{x}_t = \text{Dropout}(\mathbf{e}(i_t)), \qquad \mathbf{h}_t = \text{GRU}(\mathbf{x}_t, \mathbf{h}_{t-1}), \qquad \mathbf{u}_t = \mathbf{P}\,\text{Dropout}(\mathbf{h}_t), \tag{2}$$

with $\mathbf{e}(i) \in \mathbb{R}^{64}$, $\mathbf{h}_t \in \mathbb{R}^{128}$ and a projection $\mathbf{P} \in \mathbb{R}^{64\times128}$. Input and output embeddings are tied: the score of item j after step t is $\mathbf{u}_t^\top \mathbf{e}(j)$. The model is trained with full-softmax cross-entropy for next-item prediction at every prefix,

$$\mathcal{L} = -\sum_{u}\sum_{t=1}^{T_u-1} \log \frac{\exp(\mathbf{u}_t^\top \mathbf{e}(i_{t+1}))}{\sum_{j\in\mathcal{V}} \exp(\mathbf{u}_t^\top \mathbf{e}(j))}. \tag{3}$$

At inference, the user vector is the final projected state $\mathbf{u}_T$ and candidates are ranked by $\mathbf{u}_T^\top \mathbf{e}(c)$.

5.4 Text-aware GRU

With a median of four training items per user, most item IDs are seen in very few sequences and their embeddings are poorly estimated. The text-aware GRU (Fig. 4) keeps the network of Section 5.3 unchanged and replaces only the item representation:

$$\tilde{\mathbf{e}}(i) = \mathbf{e}(i) + \mathbf{W}\,\mathbf{t}_i, \qquad \mathbf{t}_i = \text{MiniLM}(\text{text}_i) \in \mathbb{R}^{384}, \quad \mathbf{W} \in \mathbb{R}^{64\times384}, \tag{4}$$

where $\mathbf{t}_i$ is the frozen, L2-normalised `all-MiniLM-L6-v2` embedding [13, 15] of the item's metadata, and $\mathbf{W}$ is learned. The same $\tilde{\mathbf{e}}$ is used for inputs and for the tied output layer, so two items that never co-occur can still be close because their descriptions are similar. Text features are available for 25,526 of the 25,527 vocabulary items; padding and OOV rows are zero.

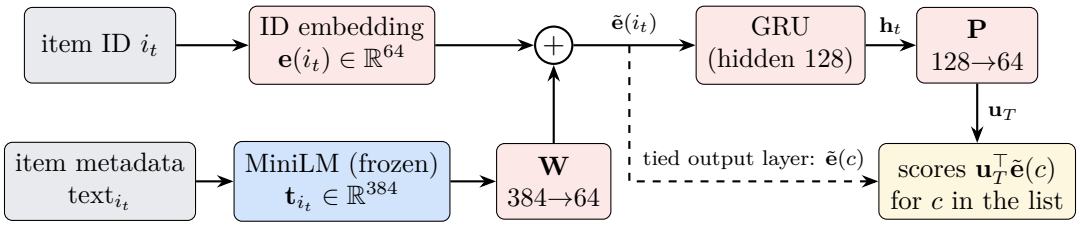


Figure 4: Text-aware GRU. The only change from the ID-only GRU is the item vector $\tilde{\mathbf{e}} = \mathbf{e} + \mathbf{W}\mathbf{t}$, shared by the input and the tied output layer.

5.5 Non-learned baselines

Content similarity. The user profile is a recency-weighted sum of the history's MiniLM vectors, $\mathbf{p}_u = \sum_{k=1}^{T} w_k \, \mathbf{t}_{i_k}$ with $w_k = \exp(-d\,(T-k))$, and candidates are ranked by $\mathbf{p}_u^\top \mathbf{t}_c$.

ItemKNN. From the binary user–item training matrix $\mathbf{X}$ we compute cosine item–item similarity $\mathrm{sim}(a,b) = \frac{(\mathbf{X}^\top \mathbf{X})_{ab}}{\sqrt{n_a n_b}}$ [14], with n_a the number of users of item a, and score candidate c as $\sum_k w_k \, \mathrm{sim}(i_k, c)$.

Popularity and random. Candidates ordered by training interaction count, or by a seeded random permutation.

LLM baselines. Following the paper, *LLM-Recency* prompts the ranker with the last ten history items, and *LLM-Vanilla-RAG* with the ten history items most similar to the mean candidate embedding; each uses one LLM call.

5.6 Late fusion

An LLM ranker outputs a permutation, not scores, so we fuse rankings. With rank $r_s(i)$ of item i under system s on a list of $n = 20$,

$$s_s(i) = 1 - \frac{r_s(i) - 1}{n - 1}, \qquad f(i) = \alpha \, s_{\mathrm{LLM}}(i) + (1 - \alpha) \, s_{\mathrm{TM}}(i), \tag{5}$$

with ties broken by NLI score, then item ID. $\alpha = 1$ reproduces ARAG exactly and $\alpha = 0$ reproduces the trained model exactly, so the trained model's contribution is exactly ablatable. $\alpha \in \{0, 0.1, \ldots, 1\}$ is chosen on the 100 validation users. We fuse ARAG with the ID-only GRU (*ARAG + GRU*, the original proposal), with the text-aware GRU and with content similarity.

6 Experimental Setup

Implementation. Python 3.11 and PyTorch; `sentence-transformers all-MiniLM-L6-v2` (384-d, normalised) with a FAISS [9] inner-product index; FastAPI and React for an interactive demonstrator. The LLM is OpenAI `gpt-4o-mini` at temperature 0 with strict JSON-schema outputs. A local `qwen3:8b` model via Ollama is supported by the same client and used for the test suite.

Hyperparameters. Table 2 lists all settings. The ID-only GRU was selected from a grid hidden $\in \{64, 128\}\times$ dropout $\in \{0.2, 0.5\}$ (best: 128/0.2, validation NDCG@5 0.299). The text-aware GRU uses the same configuration and the same 12-epoch budget, so the only difference between the two models is the item representation. Both select the checkpoint with the best validation NDCG@5 over all 94,762 validation users.

Tuning. Validation NDCG@5 for the NLI threshold was $\theta = 4$: 0.4228, $\theta = 5$: 0.4305, $\theta = 6$: 0.3993, $\theta = 7$: 0.3962 , giving $\theta^* = 5$. Table 6 shows the validation α curves.

Cost and reproducibility. Every LLM response is stored in a disk cache keyed by the hash of (model, prompt, schema). The corrected evaluation made 2,263 new calls (about \$1.06); total unique project spend across both protocol versions was 4,668 calls, 10.3 M input and 0.83 M output tokens, \$2.04. With `REPLAY_ONLY=1`, any uncached call raises an error, so the extension experiments (content baselines, text-aware GRU and new fusions) were evaluated at a guaranteed cost of \$0. A verification script recomputes every metric from the saved per-user rankings and checks dataset and list hashes against the run record.

Table 2: Hyperparameters. All selections use validation data only.

Component	Setting	Value
Both GRUs	item / hidden dimension	64 / 128
	dropout; optimiser; learning rate	0.2; Adam; 10^{-3}
	batch size; max sequence length	256; 50
	epochs (max); early-stopping patience	12; 3
	seeds	0, 1, 2
Text-aware GRU	text encoder (frozen); projection	MiniLM-L6-v2; 384→64
ARAG	NLI acceptance threshold θ^* (grid 4–7)	5
	candidates per NLI call; temperature	5; 0
Fusion	α^*: ARAG+GRU / +text-GRU / +content	0.9 / 0.3 / 0.4
Content, ItemKNN	recency decay d^* (grid 0, 0.3, 0.6, 1.0)	0.3 / 0.3
Lists	size; negatives; sampling seed	20; 19; 42
Bootstrap	resamples; interval	10,000; 95% percentile

Compute. All training ran on a laptop CPU (Apple silicon, 16 GB RAM). The ID-only GRU grid took about 1 h 50 min; the text-aware GRU takes about 5 min per seed after two memory optimisations (logits computed only at non-padded positions; chunked validation). Neither optimisation changes the loss or the original GRU's results.

7 Results

7.1 Main comparison

Table 3 reports every system on the same 100 test users, and Table 4 the non-LLM systems on all 94,762 test users. Table 5 gives the paired contrasts.

Original study. ARAG is the best system of the original study (NDCG@5 0.424). It significantly beats LLM-Recency (+0.139), LLM-Vanilla-RAG (+0.162), the ID-only GRU seeds 1 and 2 (+0.179, +0.209) and popularity (+0.315). Figure 5 visualises the comparison and Fig. 6 the hypothesis tests.

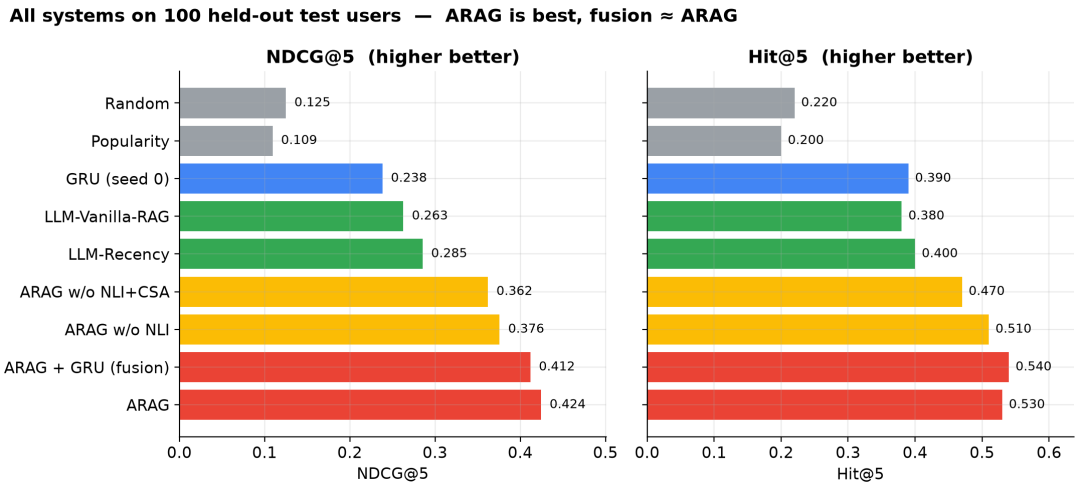


Figure 5: NDCG@5 and Hit@5 of the original study's systems on the 100 test users.

Table 3: All systems on the same 100 sampled test users (best value per column in bold). Standard deviations across users are about 0.4–0.5 for Hit and 0.3–0.45 for NDCG; see Table 5 for paired confidence intervals.

System	Hit@5	NDCG@5	Hit@10	NDCG@10	MRR
Non-learned					
Random	0.220	0.125	0.470	0.205	0.162
Popularity	0.200	0.109	0.470	0.197	0.150
Content similarity	0.590	0.439	0.820	0.510	0.429
ItemKNN	0.540	0.399	0.670	0.442	0.392
Trained sequential models					
GRU, ID only (seed 0)	0.390	0.238	0.640	0.321	0.249
GRU, ID only (seed 1)	0.450	0.246	0.640	0.306	0.230
GRU, ID only (seed 2)	0.360	0.216	0.630	0.302	0.230
Text-aware GRU (seed 0)	0.660	0.462	**0.870**	0.529	0.433
Text-aware GRU (seed 1)	0.660	**0.489**	**0.870**	**0.558**	**0.471**
Text-aware GRU (seed 2)	**0.670**	0.473	0.860	0.535	0.444
LLM baselines and ablations					
LLM-Recency	0.400	0.285	0.630	0.358	0.300
LLM-Vanilla-RAG	0.380	0.263	0.670	0.355	0.283
ARAG w/o NLI & CSA	0.470	0.362	0.630	0.414	0.372
ARAG w/o NLI	0.510	0.376	0.720	0.445	0.380
Agentic LLM and fusion					
ARAG	0.530	0.424	0.700	0.479	0.432
ARAG + GRU	0.540	0.412	0.710	0.466	0.411
ARAG + content	0.590	0.423	0.810	0.494	0.411
ARAG + text-aware GRU	0.660	0.468	**0.870**	0.537	0.443

Table 4: Non-LLM systems on all 94,762 test users (mean ± standard deviation over three seeds for the GRUs).

System	Hit@5	NDCG@5	MRR
Random	0.252	0.149	0.181
Popularity	0.213	0.128	0.166
ItemKNN	0.519	0.406	0.406
Content similarity	0.569	0.411	0.402
GRU, ID only (3 seeds)	0.422 ± 0.008	0.279 ± 0.006	0.285 ± 0.004
Text-aware GRU (3 seeds)	**0.652 ± 0.002**	**0.461 ± 0.002**	**0.435 ± 0.002**

Table 5: Paired bootstrap contrasts on the same 100 test users (ΔNDCG@5, 95% CI, 10,000 resamples). Top: original study. Bottom: extension. H1′/H2′ repeat H1/H2 with the text-aware GRU.

Contrast	ΔNDCG@5	95% CI	Sig.
H1: ARAG + GRU − ARAG	−0.013	[−0.032, +0.005]	no
H2: ARAG + GRU − GRU	+0.174	[+0.069, +0.278]	yes
ARAG − LLM-Recency	+0.139	[+0.063, +0.217]	yes
ARAG − LLM-Vanilla-RAG	+0.162	[+0.079, +0.247]	yes
ARAG − ARAG w/o NLI	+0.049	[−0.011, +0.111]	no
ARAG − ARAG w/o NLI & CSA	+0.062	[+0.016, +0.113]	yes
ARAG − GRU (seed 1)	+0.179	[+0.078, +0.282]	yes
ARAG − Popularity	+0.315	[+0.224, +0.408]	yes
Text-aware GRU − GRU	+0.224	[+0.156, +0.294]	yes
Text-aware GRU − ARAG	+0.038	[−0.067, +0.144]	no
Content similarity − ARAG	+0.014	[−0.088, +0.116]	no
ItemKNN − ARAG	−0.025	[−0.132, +0.082]	no
H1′: ARAG + text-GRU − ARAG	+0.044	[−0.035, +0.122]	no
H2′: ARAG + text-GRU − text-GRU	+0.006	[−0.046, +0.058]	no
ARAG + content − ARAG	−0.001	[−0.070, +0.066]	no

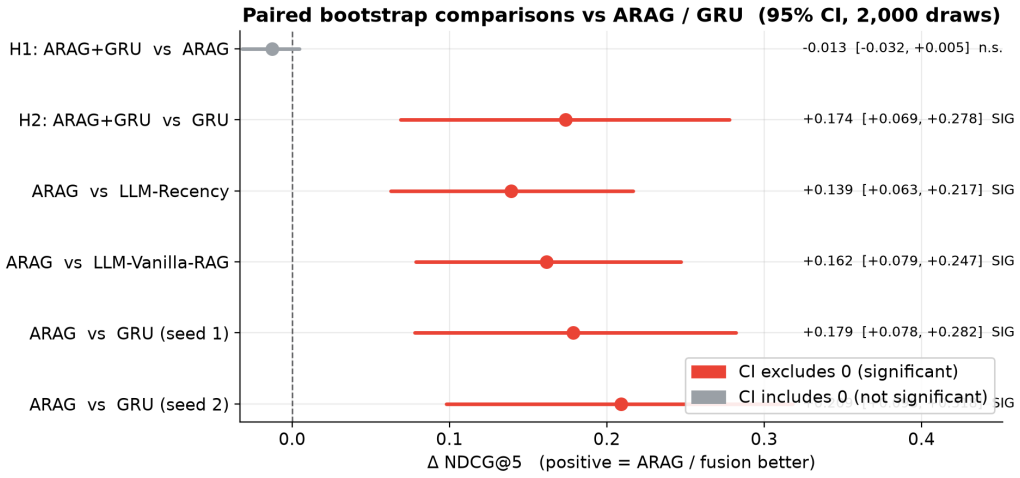


Figure 6: Forest plot of the original study's key contrasts (ΔNDCG@5, 95% CI).

7.2 RQ1: does fusing the GRU help ARAG?

H1 is not supported. At the validation-optimal $\alpha^* = 0.9$, ARAG + GRU changes NDCG@5 by -0.013 $[-0.032, +0.005]$. **H2 is supported**: the fused system beats the GRU alone by $+0.174$ $[+0.069, +0.278]$. Figure 7 and Table 6 show why: both validation and test curves rise towards the LLM end. Validation prefers $\alpha = 0.9$ by 0.0036 over $\alpha = 1$, but this reverses on test, the signature of a hyperparameter overfit on a 100-user validation set.

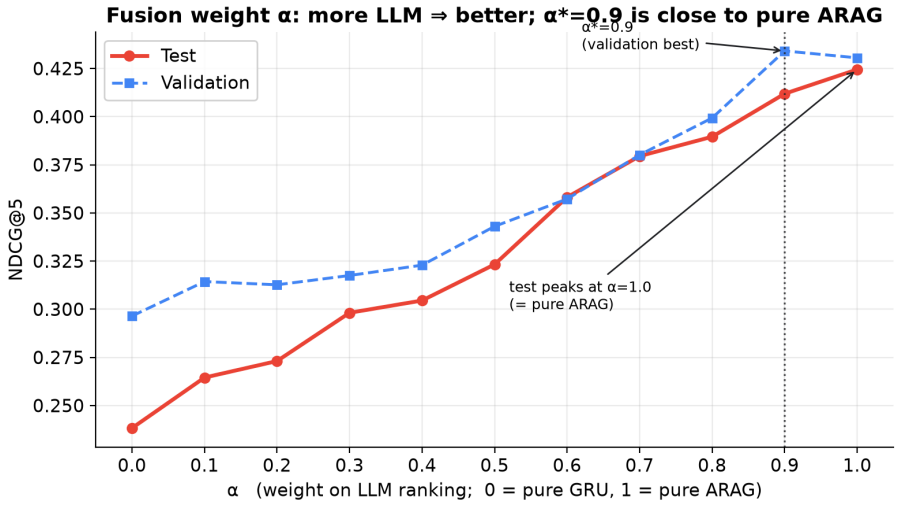


Figure 7: NDCG@5 of ARAG + GRU as a function of the fusion weight α (1 = ARAG only).

Table 6: NDCG@5 against the fusion weight α (α^* in bold where it falls on a shown grid point; for ARAG + text-GRU, $\alpha^* = 0.3$). Validation curves are on the 100 validation users.

α	0.0	0.2	0.3	0.4	0.6	0.8	0.9	1.0
ARAG + GRU, validation	0.296	0.313	0.318	0.323	0.357	0.399	**0.434**	0.431
ARAG + GRU, test	0.238	0.273	0.298	0.305	0.358	0.390	0.412	0.424
ARAG + text-GRU, validation	0.463	0.474	**0.476**	0.468	0.439	0.450	0.437	0.431
ARAG + content, validation	0.441	0.451	0.465	**0.489**	0.455	0.451	0.438	0.431

As an exploratory, post-hoc analysis on the test users (not used for any selection), Fig. 8 compares the linear rank transform of Eq. (5) with reciprocal-rank fusion and an exponential transform. None improves meaningfully on ARAG alone (0.425–0.428 vs. 0.424).

7.3 RQ2: do the paper's claims hold?

Both of the paper's baseline orderings reproduce significantly (Table 5), and the ablation staircase has the same shape as in all three of the paper's categories (Fig. 9): removing NLI and CSA significantly hurts (-0.062 $[-0.113, -0.016]$), and removing NLI alone is directionally negative but not significant (-0.049 $[-0.111, +0.011]$). Figure 10 sets our numbers next to the paper's Table 1. The absolute values are not comparable (different dataset version, category, model and candidate protocol), but ARAG's relative lift over the best baseline (+48.8% NDCG@5) is at least as large as the paper's (+25.6% to +42.1%).

7.4 RQ3: the text-aware GRU

Figure 11 shows training: the text-aware GRU passes the ID-only GRU's best validation score by epoch 3 and both non-learned baselines by epoch 9, reaching 0.476 at the 12-epoch cap (seeds: 0.476, 0.473, 0.477). On all 94,762 test users (Table 4) it reaches NDCG@5 0.461 ± 0.002 versus

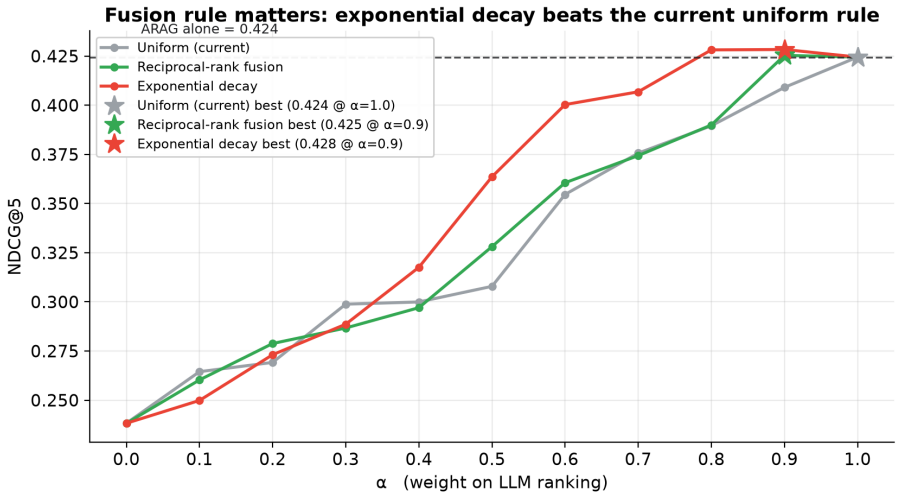


Figure 8: Exploratory comparison of rank-to-score transforms for ARAG + ID-only GRU on the test users. Shown for completeness; selecting a rule on these numbers would leak test data.

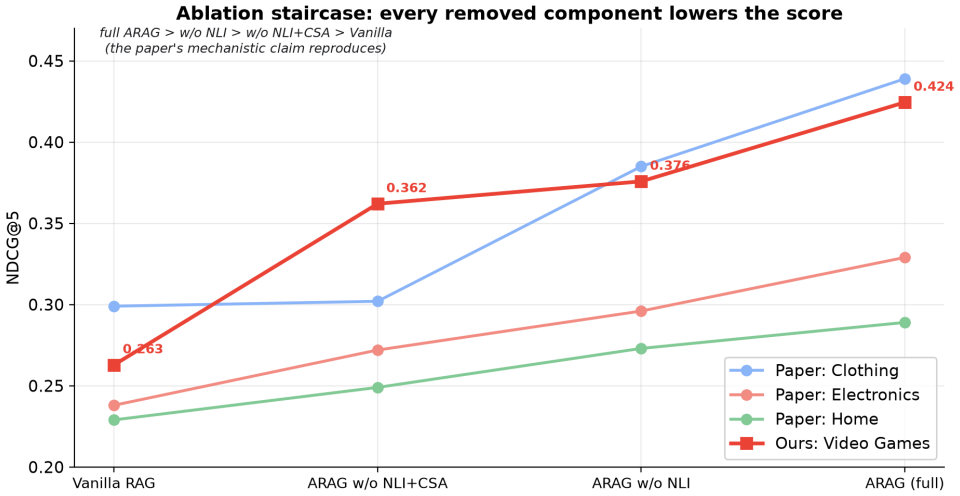


Figure 9: Ablation staircase: ours (Video Games) next to the paper's three categories.

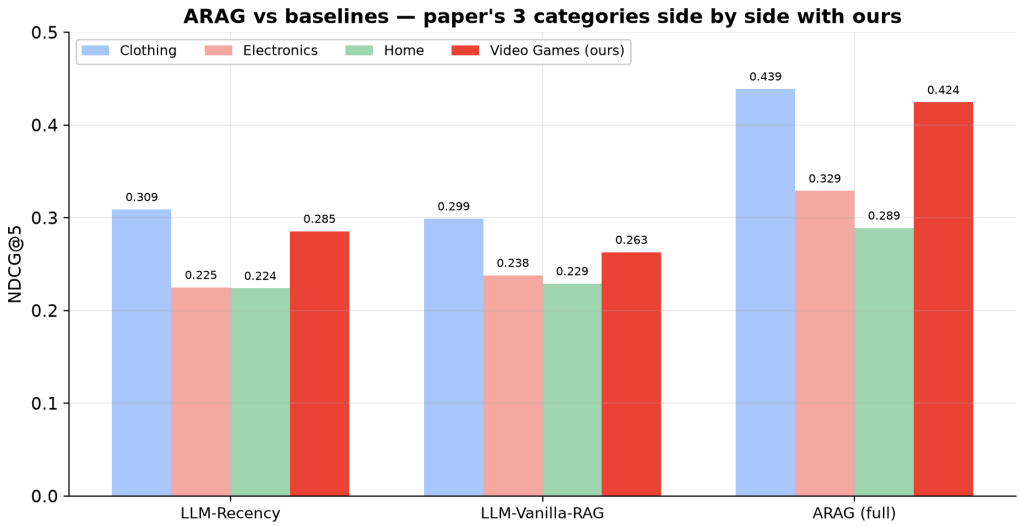


Figure 10: Our results next to the base paper's Table 1. Absolute values are shown for context only; the setups differ in dataset version, category, LLM and candidate construction.

0.279 ± 0.006 for the ID-only GRU. Paired bootstrap over all users (seed 0, 1,000 resamples) gives $+0.174$ $[+0.172, +0.176]$ over the ID-only GRU, $+0.050$ $[+0.047, +0.053]$ over content similarity and $+0.055$ $[+0.053, +0.058]$ over ItemKNN, all significant. On the 100-user sample (Fig. 12) it is numerically above ARAG for every seed ($+0.038$, $+0.065$, $+0.048$), but none of these differences is significant. Fusing it into ARAG gives the highest score in the study (0.468, $\alpha^* = 0.3$), which is not significantly different from either component (H1′: $+0.044$; H2′: $+0.006$).

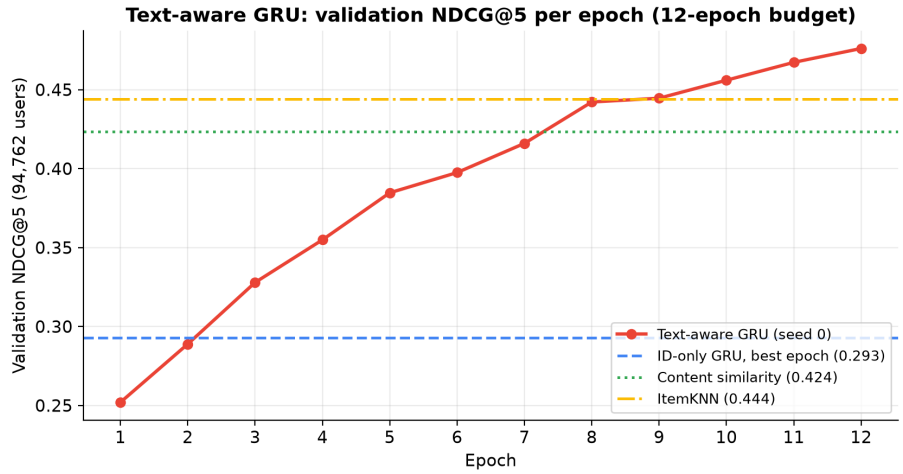


Figure 11: Validation NDCG@5 of the text-aware GRU (seed 0) per epoch, against the ID-only GRU's best epoch and the two non-learned baselines (tuned decay).

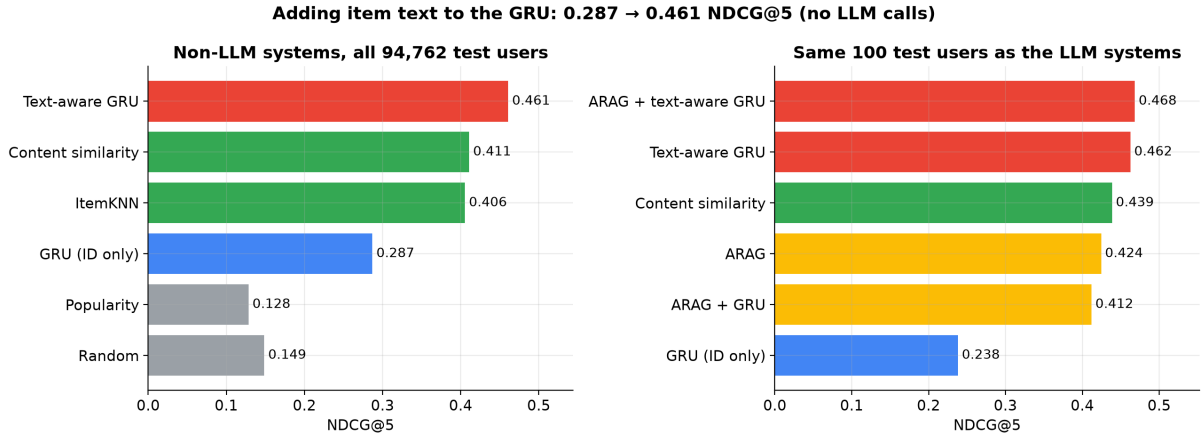


Figure 12: Extension results. Left: non-LLM systems on all 94,762 test users. Right: the same 100 users used for the LLM systems.

8 Discussion

8.1 Why the ID-only GRU adds nothing to ARAG

The ID-only GRU learns from co-occurrence over 625k training interactions, but each user contributes only a handful of IDs and most items appear in few sequences. ARAG's ranker sees the same history as text, plus the candidates' descriptions and the NLI scores; a 10% weight on a model that cannot read item content contributes no information the LLM has not already used. The null result for H1 is therefore expected once the GRU's weakness is understood.

8.2 The bottleneck was representation, not model class

The text-aware GRU changes only the item vector (Eq. (4)); the architecture, training objective, configuration and epoch budget are identical. Yet it gains +0.17–0.22 NDCG@5. With three to five interactions per user, ID embeddings are under-determined; text features let the model place rarely seen items near semantically similar ones, so the sequence model generalises across items it has seldom seen together. This mirrors findings from transferable text-based sequence models [5].

8.3 How much of ARAG's advantage is agentic reasoning?

In the original study, ARAG beat the ID-only GRU by about 0.18 NDCG@5, which could be read as evidence for agentic reasoning over learned models. The extension shows that most of that gap is explained by access to item text: an untrained content-similarity ranker is statistically tied with ARAG, and a text-aware trained model is numerically ahead of it. The agentic decomposition still matters *within* LLM ranking — ARAG significantly beats single-prompt LLM baselines and the NLI+CSA ablation — but on this protocol it does not beat a well-represented trained model. This is consistent with the broader observation that strong simple baselines are often competitive with complex recommenders [2].

8.4 Cost–quality trade-off

ARAG needs seven LLM calls per user (about $1 per 100-user evaluation with `gpt-4o-mini`, and seconds of latency per user). The text-aware GRU scores all 94,762 test users in seconds on a CPU at no marginal cost, and trains in about 15 minutes for three seeds. On this protocol it is the better operating point.

8.5 A methodological lesson: candidate construction can manufacture conclusions

An earlier version of the study sampled negatives uniformly from all 137,260 catalogue items with text. Because 81% of those items have no interactions, a mean of 15.5 of the 19 negatives were unknown to popularity (all tied at zero) and to the GRU (all mapped to one OOV embedding). The effective task for those systems was a choice among about 4.5 items. Popularity reached Hit@5 = 0.97, the GRU NDCG@5 = 0.82, and fusion appeared to improve ARAG by +0.349 ("H1 supported"). The flaw was found in review; the lists were rebuilt with popularity-matched in-vocabulary negatives, every system was re-evaluated and the GRU retrained. The conclusion reversed (Fig. 13). The general point, also made by Krichene and Rendle [11], is that sampled-candidate protocols must be checked for shortcuts that some systems can exploit and others cannot.

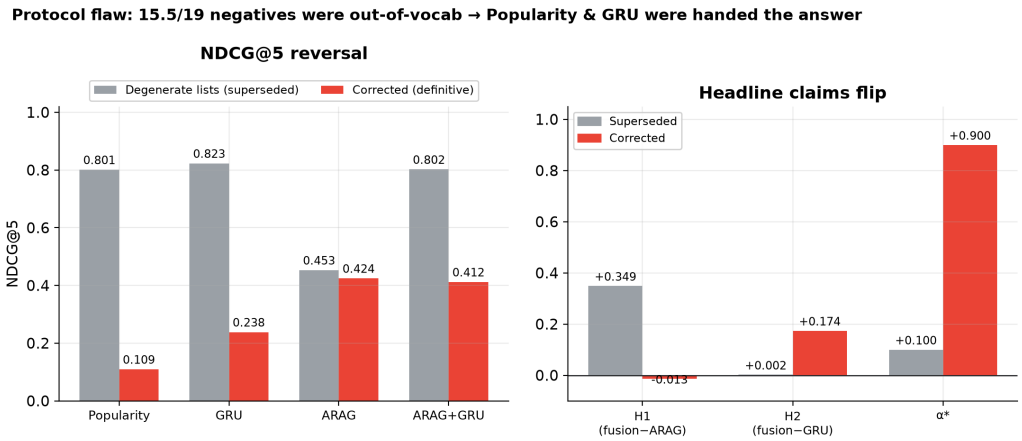


Figure 13: The original (degenerate) candidate lists versus the corrected lists. Under the degenerate lists, popularity-like systems appeared dominant and H1 appeared supported.

9 Threats to Validity

1. **Sample size.** Only 100 users are evaluated with the LLM (cost). Confidence intervals on NDCG@5 differences are roughly ±0.08–0.10, so the top systems (ARAG, content similarity, text-aware GRU and their fusions) cannot be separated. About 300 users (≈ $3) would be needed to resolve differences of ∼ 0.05.

2. **Sampled negatives.** Popularity-matched negatives remove the popularity shortcut but are easier than candidates mined by a real retriever. Results describe ranking quality on hard 20-item lists, not end-to-end retrieval quality.

3. **Slight tilt of the negatives.** Popularity scores slightly below random (Hit@5 0.213 vs. 0.252 on all users), so negatives are on average a little more popular than targets. This slightly disadvantages frequency-sensitive models, including the GRUs. The effect (∼ 0.04 Hit@5) is much smaller than the gaps we interpret.

4. **Leave-last-out leakage.** Other users' later interactions can appear in the training data of the GRUs and ItemKNN. This is inherent to the official split and affects those systems equally; content similarity does not use other users' data.

5. **Training budget.** The text-aware GRU was still improving at the 12-epoch cap. The cap was kept to match the ID-only GRU's budget, so the reported gain is, if anything, conservative.

6. **Tuning noise.** α and θ are tuned on 100 validation users; the validation and test α curves disagree near their maxima.

7. **Scope.** One category, one dataset, one LLM. The study is ARAG-inspired, not an exact reproduction: the paper does not publish prompts, its threshold or its candidate construction, and it used `gpt-3.5-turbo` on the 2016 Amazon data.

8. **Offline metrics only.** No online or human evaluation; one held-out item per user defines relevance.

10 Conclusion and Future Work

We built an ARAG-style agentic recommender and evaluated it against, and fused with, a trained GRU under a protocol that isolates ranking and tests every contrast statistically. Three findings stand out. First, fusing an ID-only GRU into ARAG does not help (H1 fails), while the agentic

pipeline significantly beats simpler LLM baselines and its NLI+CSA ablation, reproducing the base paper's claims on a new category and model. Second, the GRU's weakness is a representation problem: adding frozen sentence embeddings of item metadata to its item vectors lifts it from NDCG@5 0.279 to 0.461 on all test users, past content and collaborative baselines, and numerically past ARAG, at zero LLM cost. Third, the construction of sampled candidate lists can create or erase entire conclusions, so it must be audited.

Future work: a larger LLM-evaluated sample to separate the top systems; negatives mined by a retriever; longer training and a Transformer sequence model (SASRec) with the same text-aware item representation; feeding the trained model's scores to the IRA agent as an explicit feature instead of fusing afterwards; and replication on further categories.

References

[1] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of EMNLP*, 2014.

[2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of ACM RecSys*, 2019.

[3] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[4] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk. Session-based recommendations with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*, 2016.

[5] Y. Hou, S. Mu, W. X. Zhao, Y. Li, B. Ding, and J.-R. Wen. Towards universal sequence representation learning for recommender systems. In *Proceedings of ACM SIGKDD*, 2022.

[6] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley. Bridging language and items for retrieval and recommendation. *arXiv preprint arXiv:2403.03952*, 2024.

[7] Y. Hou, J. Zhang, Z. Lin, H. Lu, R. Xie, J. McAuley, and W. X. Zhao. Large language models are zero-shot rankers for recommender systems. In *Proceedings of ECIR*, 2024.

[8] K. Järvelin and J. Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

[9] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[10] W.-C. Kang and J. McAuley. Self-attentive sequential recommendation. In *Proceedings of IEEE ICDM*, 2018.

[11] W. Krichene and S. Rendle. On sampled metrics for item recommendation. In *Proceedings of ACM SIGKDD*, 2020.

[12] R. Y. Maragheh, P. Vadla, P. Gupta, K. Zhao, A. Inan, K. Yao, J. Xu, P. Kanumala, J. Cho, and S. Kumar. ARAG: Agentic retrieval augmented generation for personalized recommendation. *arXiv preprint arXiv:2506.21931*, 2025.

[13] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.

[14] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of WWW*, 2001.

[15] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

A Worked Example

A real test user (`AEDTQXNMWBEI7UECETV5JVUEWNDQ`) with a four-item history: *The Legend of Zelda: Breath of the Wild* (Wii U), *Need for Speed Most Wanted U* (Wii U), *Hot Wheels World's Best Driver* and *LEGO Marvel* (PC). The frozen list contains twenty plausible gaming products — for example *Demon's Souls* (PS3), *New Super Mario Bros. U*, a Luigi amiibo, *Ghost of Tsushima* (PS4), PS4 controller accessories and two gaming headsets. The held-out target is an *ARKARTECH Gaming Headset*. Every negative is a known, reviewed product, illustrating that the corrected lists contain no trivial zero-interaction decoys: a ranker must infer from four console-game purchases that an accessory such as a headset is a plausible next purchase.

B Errata: the Superseded Run

Table 7: Original (degenerate lists) versus corrected results.

Quantity	Original lists	Corrected lists
Unknown negatives per list (of 19)	15.5	0
Popularity NDCG@5	0.801	0.109
GRU NDCG@5 (all users)	0.823	0.279
ARAG NDCG@5	0.453	0.424
α^*	0.1	0.9
H1 (ARAG + GRU − ARAG)	+0.349 "significant"	−0.013 n.s.
H2 (ARAG + GRU − GRU)	+0.002 n.s.	+0.174 significant
ARAG vs. Recency / Vanilla-RAG	n.s.	both significant
Best system	GRU / fused	ARAG (text-aware GRU in the extension)

C Reproducibility

All code, configuration and results are in the project repository. The full pipeline is:

```
make setup                       # environment
make data lists index            # dataset, frozen lists, MiniLM + FAISS
make gru gru-text                # ID-only GRU (grid + 3 seeds), text-aware GRU (3 seeds)
make eval                        # all systems (LLM calls disk-cached)
make report verify               # results/, and re-derive every metric from raw predictions
python scripts/make_figures.py   # figures
python scripts/make_paper_tables.py   # the LaTeX tables of this report
```

A zero-cost replay of the full evaluation is `REPLAY_ONLY=1 OPENAI_API_KEY=replay-only-dummy make eval`; any LLM call missing from the cache raises an error instead of reaching the API. Every table in Section 7 is generated from `results/metrics.json` by `scripts/make_paper_tables.py`.